الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء التاسع: مكتبة Scikit-learn

الفصول ١ - ٦: من التعريف بالمكتبة حتى الدوال الأساسية

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة مهمة 4
الفصل الأول: التعريف بمكتبة Scikit-learn 5
١.١ نشأتها والمشكلة التي حلّتها 5
١.٢ أهم الاستخدامات والشركات 5
١.٣ المميزات والعيوب 5
١.٤ متى تستخدمها ومتى لا 5
١.٥ أشهر البدائل والمكمّلات 6
ملخص الفصل 6
الفصل الثاني: التثبيت والإعداد 7
٢.١ أفضل أسلوب استيراد 7
٢.٢ تنظيم المشروع 7
٢.٣ حفظ واستعادة النماذج 7
أفضل الممارسات 7
الفصل الثالث: فلسفة التصميم 8
٣.١ Consistency: المبدأ الأعلى فوق كل شيء 8
٣.٢ Estimator: المفهوم الأساسي الموحّد 8
٣.٣ لماذا هذا التصميم تحديدًا؟ 8
٣.٤ فئات أساسية (Base Classes) خلف الكواليس 8
٣.٥ فلسفة عدم التحيّز الحسابي (Stateless حتى fit) 9
ملخص الفصل 9
الفصل الرابع: المفاهيم الأساسية 10
٤.١ دورة العمل القياسية: fit / predict / transform 10
٤.٢ Transformer وfit_transform 10
٤.٣ Pipeline: تركيب Estimators متعددة كوحدة واحدة 10
٤.٤ مجموعات البيانات المدمجة (Datasets) 10
٤.٥ أدوات اختيار النموذج (Model Selection) 11
الأخطاء الشائعة 11
تمارين عملية 11
حلول مقترحة (تلميحات) 11
الفصل الخامس: الكائنات وواجهات البرمجة 12
٥.١ get_params وset_params 12
٥.٢ اصطلاح الشرطة السفلية اللاحقة (Trailing Underscore) 12

٥.٣ ColumnTransformer: تطبيق تحويلات مختلفة على أعمدة مختلفة 12
٥.٤ نموذج التنفيذ: Eager وValidation صارم عند fit 12
أفضل الممارسات 12
الفصل السادس: أهم الدوال والأدوات 14
٦.١ أدوات المعالجة المسبقة (Preprocessing) 14
٦.٢ أدوات اختيار الخصائص (Feature Selection) 14
٦.٣ مقاييس التقييم (Metrics) 14
٦.٤ أدوات التحقق من الصحة (Input Validation Utilities) 14
٦.٥ كيفية قراءة توثيق Scikit-learn بكفاءة 15
الأخطاء الشائعة 15
الأسئلة الشائعة 15
تمارين عملية 15
حلول مقترحة (تلميحات) 15
مراجع وروابط رسمية 16
خاتمة الجزء الأول من Scikit-learn 16

مقدمة مهمة

هذا الجزء يتعامل مع Scikit-learn حصرًا كإطار عمل هندسي (Framework) له واجهة برمجية موحّدة، وليس كمصدر لتعلّم خوارزميات التعلّم الآلي. لن نشرح كيف يعمل الانحدار الخطي رياضيًا، ولا كيف تُبنى شجرة قرار داخليًا، بل سنركّز بالكامل على: كيف صُممت واجهة Scikit-learn البرمجية، لماذا صُممت بهذا الشكل، وكيف تستخدمها باحترافية في أي مشروع بغض النظر عن الخوارزمية المحددة المستخدمة داخل كل Estimator.

الفصل الأول: التعريف بمكتبة Scikit-learn

١.١ نشأتها والمشكلة التي حلّتها

بدأ مشروع Scikit-learn عام ٢٠٠٧ كمشروع Google Summer of Code بواسطة David Cournapeau، ثم أعاد فريق من INRIA (المعهد الفرنسي للبحث في المعلوماتية) بقيادة Fabian Pedregosa بناءه وتطويره بشكل جوهري، وصدر أول إصدار مستقر عام ٢٠١٠. المشكلة التي حلّتها: قبلها كانت خوارزميات التعلّم الآلي في بايثون متناثرة في مكتبات وتطبيقات منفصلة بواجهات غير متسقة، وكان على الباحث تعلّم واجهة برمجية جديدة لكل خوارزمية. وحّدت Scikit-learn كل هذا التنوع خلف واجهة برمجية واحدة متسقة تمامًا (Estimator API)، بحيث يصبح تبديل خوارزمية بأخرى غالبًا مجرد تغيير سطر استيراد واحد.

١.٢ أهم الاستخدامات والشركات

- بناء نماذج تصنيف وانحدار وتجميع للبيانات الجدولية.
- معالجة وهندسة الخصائص (Feature Engineering) قبل التدريب.
- خطوط أنابيب تعلّم آلي كاملة عبر Pipeline من التحضير حتى التقييم.
- الأساس القياسي لأي مقرر أو كتاب تعليمي في التعلّم الآلي التقليدي (غير العميق).

تعتمد عليها شركات من كل الأحجام لبناء نماذج تعلّم آلي تقليدية (غير عصبية عميقة)، وهي غالبًا نقطة البداية القياسية لأي مشروع تعلّم آلي قبل التفكير في حلول أكثر تعقيدًا مثل الشبكات العصبية العميقة.

١.٣ المميزات والعيوب

الجانب	التفاصيل
مميزات	واجهة برمجية موحّدة ومتسقة تمامًا (fit/predict/transform)، توثيق ممتاز، تكامل عميق مع NumPy وPandas، تغطية واسعة لخوارزميات التعلّم الآلي التقليدي
عيوب	لا تدعم الشبكات العصبية العميقة أو GPU أصلاً، أداء محدود على بيانات ضخمة جدًا مقارنة بأطر متخصصة، بعض الخوارزميات ذات نسخة واحدة فقط (لا بدائل معمارية متعددة)
حدود	مصممة للبيانات الجدولية المتوسطة الحجم؛ ليست الخيار الأنسب للصور أو النصوص المعقدة أو البيانات الضخمة الموزّعة

١.٤ متى تستخدمها ومتى لا

القرار	الحالة
Scikit-learn	بيانات جدولية متوسطة الحجم، تصنيف/انحدار/تجميع تقليدي
PyTorch أو TensorFlow/Keras	صور، نصوص معقدة، أو حاجة لشبكات عصبية عميقة
Dask-ML أو Spark MLlib	بيانات ضخمة جدًا لا تتسع بالذاكرة
أدوات Scikit-learn للمعالجة المسبقة (Preprocessing) ما زالت مفيدة	تحضير بيانات وخطوط أنابيب قبل أي نموذج، حتى لو النموذج نفسه من مكتبة أخرى

١.٥ أشهر البدائل والمكمّلات

- XGBoost / LightGBM / CatBoost: مكتبات متخصصة لخوارزميات Gradient Boosting بأداء أعلى، وتتبنى واجهة متوافقة جزئيًا مع Scikit-learn API.
- PyTorch / TensorFlow: للتعلّم العميق والشبكات العصبية.
- Dask-ML: لتوسيع بعض واجهات Scikit-learn لتعمل على بيانات موزّعة أكبر من الذاكرة.

ملخص الفصل

- Scikit-learn وُلدت لتوحيد واجهات خوارزميات التعلّم الآلي المتناثرة خلف Estimator API واحد ومتسق.
- هي الخيار القياسي للتعلّم الآلي التقليدي على بيانات جدولية؛ ليست مصممة للتعلّم العميق أو البيانات الضخمة جدًا.
- توثيقها واتساق واجهتها يجعلانها أداة تعليمية ومهنية ممتازة لفهم أي مشروع تعلّم آلي تقليدي مفتوح المصدر.

الفصل الثاني: التثبيت والإعداد

```
pip install scikit-learn
conda install -c conda-forge scikit-learn
```

٢.١ أفضل أسلوب استيراد

```
# الاسم على PyPI هو scikit-learn، لكن الاستيراد الفعلي دائمًا sklearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
```

ملاحظة مهمة للمبتدئين: اسم حزمة pip هو scikit-learn (بشرطات)، لكن اسم الوحدة المستوردة فعليًا في الكود هو sklearn دون شرطات — وهذا الفرق مصدر ارتباك شائع جدًا.

٢.٢ تنظيم المشروع

```
project/
├── data/
├── src/
│   ├── features.py     # هندسة الخصائص
│   ├── pipeline.py      # بناء Pipeline كامل
│   └── evaluate.py      # مقاييس التقييم
├── models/               # نماذج محفوظة (joblib)
└── notebooks/
```

٢.٣ حفظ واستعادة النماذج

```
import joblib

joblib.dump(pipeline, 'models/pipeline_v1.joblib')
loaded = joblib.load('models/pipeline_v1.joblib')
```

joblib هي الأداة القياسية لحفظ كائنات Scikit-learn (خاصة تلك المحتوية مصفوفات NumPy كبيرة)، وهي أكفأ من pickle العادي لهذا الغرض تحديدًا.

أفضل الممارسات

- تذكّر أن اسم الاستيراد sklearn وليس scikit_learn أو scikit-learn.
- استخدم joblib لحفظ النماذج والـ Pipelines، وليس pickle مباشرة.
- ثبّت إصدار محدد من scikit-learn في requirements.txt، فقد تتغير القيم الافتراضية بين الإصدارات.

الفصل الثالث: فلسفة التصميم

٣.١ Consistency: المبدأ الأعلى فوق كل شيء

الفلسفة المركزية الوحيدة التي يمكن تلخيص كل تصميم Scikit-learn تحتها هي: الاتساق (Consistency). كل خوارزمية — سواء كانت تصنيفًا أو انحدارًا أو تجميعًا أو تقليل أبعاد — تُعامَل عبر نفس الأنماط البرمجية القليلة والمتكررة: fit، predict، transform، score. هذا يعني أنك إذا أتقنت التعامل مع خوارزمية واحدة، فأنت عمليًا تعرف كيف تتعامل مع عشرات الخوارزميات الأخرى في المكتبة، لأن الفرق بينها غالبًا هو اسم الفئة المستوردة فقط لا أسلوب الاستخدام.

٣.٢ Estimator: المفهوم الأساسي الموحّد

كل 'شيء يتعلّم من البيانات' في Scikit-learn — نموذج تصنيف، محوّل بيانات، حتى أداة اختيار خصائص — هو Estimator. كل Estimator يطبّق واجهة موحّدة تتمحور حول دالة fit(X, y=None) التي 'تتعلّم' من البيانات وتُخزّن ما تعلّمته كخصائص داخلية تنتهي أسماؤها بشرطة سفلية (مثل model.coef_).

نوع Estimator	الطرق الأساسية	مثال
Predictor (نموذج تنبؤي)	fit(X, y) ثم predict(X)	LogisticRegression, RandomForestClassifier
Transformer (محوّل بيانات)	fit(X) ثم transform(X)، أو fit_transform(X) دفعة واحدة	StandardScaler, PCA
Clusterer (نموذج تجميع)	fit(X) ثم predict(X) أو fit_predict(X)	KMeans

٣.٣ لماذا هذا التصميم تحديدًا؟

توحيد الواجهة بهذا الشكل الصارم يخدم هدفًا هندسيًا محددًا: قابلية التبديل (Interchangeability). بما أن كل Estimator يتبع نفس العقد البرمجي (Contract)، يمكن لأدوات مثل Pipeline وGridSearchCV أن تتعامل مع أي Estimator دون معرفة تفاصيله الداخلية إطلاقًا — تمامًا كما يسمح مبدأ التعددية الشكلية (Polymorphism) في البرمجة الكائنية بمعاملة كائنات مختلفة عبر واجهة موحّدة.

٣.٤ فئات أساسية (Base Classes) خلف الكواليس

داخليًا، تُبنى كل Estimators فوق فئات أساسية مثل BaseEstimator وTransformerMixin وClassifierMixin، والتي توفّر سلوكًا مشتركًا جاهزًا (مثل get_params وset_params لإدارة المعاملات، أو score لتقييم النموذج تلقائيًا)، بحيث لا يحتاج كل Estimator جديد لإعادة تطبيق هذا السلوك المشترك من الصفر.

```
from sklearn.base import BaseEstimator, TransformerMixin

class MyCustomTransformer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        return X * 2   # مثال بسيط لمنطق تحويل مخصص
```

هذا النمط — وراثة BaseEstimator وTransformerMixin — هو الطريقة الرسمية الموصى بها لبناء محوّلات بيانات مخصصة تتوافق تمامًا مع بقية نظام Scikit-learn (بما في ذلك العمل داخل Pipeline).

٣.٥ فلسفة عدم التحيّز الحسابي (Stateless حتى fit)

مبدأ تصميمي دقيق لكنه جوهري: لا يُسمح لأي Estimator بتغيير حالته الداخلية إلا داخل دالة fit تحديدًا؛ حتى المُنشئ (Constructor، أي __init__) لا يجب أن يقوم بأي معالجة أو تحقق من البيانات، بل يكتفي بتخزين المعاملات كما هي. هذا يضمن سلوكًا يمكن التنبؤ به تمامًا عند استنساخ Estimator (عبر clone) أو إعادة استخدامه في GridSearchCV.

ملخص الفصل

- الفلسفة المحورية الوحيدة هي الاتساق: كل Estimator يتبع نفس العقد البرمجي (fit/predict/transform).
  - Estimator هو المفهوم الموحّد لأي 'شيء يتعلّم من البيانات'، سواء كان نموذجًا تنبؤيًا أو محوّل بيانات.
- هذا التصميم يمكّن أدوات عليا مثل Pipeline وGridSearchCV من التعامل مع أي Estimator دون معرفة تفاصيله الداخلية.

الفصل الرابع: المفاهيم الأساسية

٤.١ دورة العمل القياسية: fit / predict / transform

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()   # 1. الإنشاء بمعاملات (Hyperparameters)
model.fit(X_train, y_train)    # 2. التعلّم من بيانات التدريب
predictions = model.predict(X_test)   # 3. التنبؤ على بيانات جديدة
accuracy = model.score(X_test, y_test)  # 4. تقييم سريع مدمج
```

٤.٢ Transformer وfit_transform

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)                    # يتعلّم المتوسط والانحراف المعياري من التدريب فقط
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)   # يُطبّق بنفس القيم المتعلَّمة من التدريب

# اختصار شائع لبيانات التدريب فقط
X_train_scaled = scaler.fit_transform(X_train)
```

مبدأ جوهري يجب فهمه بعمق: fit تُستدعى فقط على بيانات التدريب أبدًا على بيانات الاختبار، لتفادي 'تسرّب البيانات' (Data Leakage) — وهو استخدام معلومات من بيانات الاختبار (حتى إحصائيًا مثل المتوسط) أثناء تدريب النموذج، مما يجعل تقييم الأداء لاحقًا مضلِّلاً ومتفائلاً زيادة عن الواقع.

٤.٣ Pipeline: تركيب Estimators متعددة كوحدة واحدة

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression()),
])

pipe.fit(X_train, y_train)       # ينفّذ fit_transform على scaler ثم fit على model تلقائيًا
pipe.predict(X_test)             # ينفّذ transform على scaler ثم predict على model تلقائيًا
```

Pipeline هو تطبيق مباشر ومباشِر لفلسفة الاتساق: بما أن كل خطوة تتبع نفس العقد البرمجي (fit/transform)، يمكن لـ Pipeline تسلسلها تلقائيًا معاملًا Pipeline بأكمله كـ Estimator واحد متكامل، مما يمنع تسرّب البيانات تلقائيًا ويبسّط الكود بشكل كبير.

٤.٤ مجموعات البيانات المدمجة (Datasets)

```
from sklearn.datasets import load_iris, make_classification
```

```
data = load_iris()                # بيانات حقيقية صغيرة للتجريب السريع
X, y = data.data, data.target

X, y = make_classification(n_samples=1000, n_features=10)  # بيانات اصطناعية للاختبار
```

٤.٥ أدوات اختيار النموذج (Model Selection)

```
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scores = cross_val_score(model, X_train, y_train, cv=5)   # تقييم عبر 5 طيّات

grid = GridSearchCV(model, param_grid={'C': [0.1, 1, 10]}, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_)
```

GridSearchCV هو مثال ممتاز آخر على قوة Estimator API الموحّد: هو نفسه Estimator (يملك fit وpredict)، لكنه 'يغلّف' Estimator آخر بداخله ويبحث آليًا عن أفضل معاملات (Hyperparameters) عبر التحقق المتقاطع (Cross-Validation).

الأخطاء الشائعة

- استدعاء fit_transform على بيانات الاختبار بدل transform فقط، مما يسبب تسرّب بيانات صامت.
- نسيان أن train_test_split تُنتج تقسيمًا عشوائيًا مختلفًا في كل تشغيل ما لم تُحدَّد random_state، مما يجعل النتائج غير قابلة لإعادة الإنتاج.

تمارين عملية

١. حمّل بيانات load_iris، قسّمها train/test، درّب LogisticRegression، واحسب score على بيانات الاختبار.

٢. ابنِ Pipeline يجمع StandardScaler وLogisticRegression معًا، ودرّبه مباشرة على بيانات غير مُطبَّعة.

٣. استخدم GridSearchCV للبحث عن أفضل قيمة C لنموذج LogisticRegression عبر ٣ قيم مختلفة.

حلول مقترحة (تلميحات)

١. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

٢. Pipeline([('scaler', StandardScaler()), ('model', LogisticRegression())]).fit(X_train, y_train)

٣. GridSearchCV(LogisticRegression(), {'C':[0.1,1,10]}, cv=5).fit(X_train, y_train)

الفصل الخامس: الكائنات وواجهات البرمجة

٥.١ get_params وset_params

```
model = LogisticRegression(C=1.0, max_iter=100)
print(model.get_params())   # {'C': 1.0, 'max_iter': 100, ...}
model.set_params(C=10.0)    # تغيير معامل بعد الإنشاء
```

كل Estimator يطبّق get_params وset_params تلقائيًا بمجرد وراثة BaseEstimator بشرط تخزين كل معامل من __init__ كخاصية بنفس الاسم تمامًا دون تعديل — وهذا شرط توثيقي صارم في Scikit-learn يتيح لأدوات مثل GridSearchCV التعامل مع أي Estimator بشكل عام تمامًا.

٥.٢ اصطلاح الشرطة السفلية اللاحقة (Trailing Underscore)

اصطلاح تسمية صارم في كل المكتبة: أي خاصية تُحسَب أو 'تُتعلَّم' من البيانات أثناء fit تنتهي دائمًا بشرطة سفلية، للتفريق الواضح بينها وبين معاملات الإعداد الأولية (Hyperparameters) الممرَّرة في __init__.

```
model.fit(X_train, y_train)
print(model.coef_)          # مُتعلَّم من البيانات — شرطة سفلية لاحقة
print(model.intercept_)     # مُتعلَّم من البيانات
print(model.C)              # معامل إعداد أولي — بلا شرطة سفلية
```

٥.٣ ColumnTransformer: تطبيق تحويلات مختلفة على أعمدة مختلفة

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), ['age', 'salary']),
    ('cat', OneHotEncoder(), ['city', 'department']),
])

pipe = Pipeline([('preprocess', preprocessor), ('model', LogisticRegression())])
```

ColumnTransformer نفسه Estimator يتبع نفس العقد البرمجي، ويحل مشكلة شائعة جدًا: البيانات الجدولية الحقيقية دائمًا مزيج من أعمدة عددية وفئوية تحتاج معالجات مختلفة تمامًا، وهو يطبّق كل معالجة على أعمدتها المحددة داخل Pipeline واحد متسق.

٥.٤ نموذج التنفيذ: Eager وValidation صارم عند fit

مثل NumPy وPandas، تتبع Scikit-learn نموذج تنفيذ فوري بالكامل (Eager). لكن ما يميّزها هو تطبيق تحقق صارم من صحة المدخلات (Input Validation) في بداية كل fit وpredict — فحص الأبعاد، أنواع البيانات، القيم المفقودة — قبل تنفيذ أي منطق حسابي، مما ينتج رسائل خطأ واضحة نسبيًا في حال كانت البيانات غير متوافقة.

أفضل الممارسات

- استخدم ColumnTransformer دائمًا عند وجود أعمدة عددية وفئوية مختلطة بدل معالجتها يدويًا خارج Pipeline.
- لا تُعدّل أي معامل __init__ داخل دالة أخرى غير set_params لتبقى متوافقًا مع نظام Scikit-learn الداخلي.
- افحص الخصائص المنتهية بشرطة سفلية (feature_importances_, coef_...) لفهم ما تعلّمه النموذج، لا لتفسير كيفية عمل الخوارزمية رياضيًا.

الفصل السادس: أهم الدوال والأدوات

٦.١ أدوات المعالجة المسبقة (Preprocessing)

الأداة	الاستخدام
StandardScaler	توحيد القياس: متوسط ٠ وانحراف معياري ١
MinMaxScaler	تحجيم القيم إلى نطاق ثابت (عادة [٠،١])
OneHotEncoder	تحويل أعمدة فئوية إلى أعمدة ثنائية (١/٠)
OrdinalEncoder	ترميز فئات مرتّبة كأرقام صحيحة متسلسلة
SimpleImputer	تعبئة القيم المفقودة بإستراتيجية محددة (متوسط، وسيط، أكثر تكرارًا)

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
X_filled = imputer.fit_transform(X_with_missing)
```

٦.٢ أدوات اختيار الخصائص (Feature Selection)

```
from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(score_func=f_classif, k=5)
X_selected = selector.fit_transform(X_train, y_train)
```

٦.٣ مقاييس التقييم (Metrics)

الدالة	نوع المهمة	الاستخدام
accuracy_score	تصنيف	نسبة التنبؤات الصحيحة
/ precision_score / recall_score f1_score	تصنيف	مقاييس أدق عند عدم توازن الفئات
confusion_matrix	تصنيف	جدول تفصيلي للتنبؤات الصحيحة والخاطئة
mean_squared_error / r2_score	انحدار	قياس دقة التنبؤ بقيم مستمرة

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = pipe.predict(X_test)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

٦.٤ أدوات التحقق من الصحة (Input Validation Utilities)

```
from sklearn.utils.validation import check_is_fitted, check_array

def transform(self, X):
    check_is_fitted(self)          # يرفع خطأ واضح إن لم يُستدعَ fit بعد
```

```
    X = check_array(X)              # يتحقق من صحة شكل ونوع المدخلات
    return X * self.factor_
```

هذه الأدوات هي ما يفسّر رسائل الخطأ الواضحة نسبيًا في Scikit-learn مقارنة بمكتبات أخرى؛ فهي تتحقق من المدخلات مبكرًا وبشكل موحّد عبر كل المكتبة بدل ترك كل Estimator يتحقق بطريقته الخاصة.

٦.٥ كيفية قراءة توثيق Scikit-learn بكفاءة

- كل صفحة API Reference لأي Estimator تتبع نفس البنية: Parameters (معاملات __init__)، Attributes (الخصائص المنتهية بشرطة سفلية بعد fit)، Methods (fit/predict/transform وغيرها).
- قسم 'Examples' في نهاية كل صفحة توثيق غالبًا أسرع طريقة لفهم الاستخدام العملي دون قراءة الشرح الكامل.
- دليل المستخدم (User Guide) المنفصل عن API Reference يشرح السياق والخلفية النظرية بعمق أكبر لمن يحتاجها.

الأخطاء الشائعة

- استخدام accuracy_score وحدها كمقياس تقييم عند بيانات غير متوازنة (Imbalanced Classes)، بينما precision وrecall وf1 أكثر دلالة في تلك الحالة.
- تطبيق SimpleImputer أو StandardScaler على كامل البيانات قبل تقسيمها train/test بدل تدريبها على التدريب فقط، مما يسبب تسرّب بيانات.

الأسئلة الشائعة

س: هل يجب أن أفهم رياضيات كل خوارزمية لاستخدام Scikit-learn باحتراف؟

ج: لفهم متى تختار خوارزمية دون أخرى وكيف تفسّر نتائجها بعمق، نعم يفيد فهم الأساسيات النظرية؛ لكن لاستخدام الواجهة البرمجية باحتراف وبناء خطوط أنابيب سليمة خالية من تسرّب البيانات، هذا الفصل وحده كافٍ.

س: ما الفرق بين transform وfit_transform من ناحية الأداء؟

ج: fit_transform ليست دائمًا مجرد استدعاء الاثنين بالتتابع؛ بعض المحوّلات (مثل بعض خوارزميات تقليل الأبعاد) تنفّذها بشكل محسوب مجتمعًا أكفأ من استدعاء fit ثم transform منفصلين.

تمارين عملية

١. ابنِ Pipeline بمعالجة SimpleImputer ثم StandardScaler ثم LogisticRegression، ودرّبه على بيانات تحتوي قيمًا مفقودة.

٢. احسب classification_report وconfusion_matrix لنموذج تصنيف مدرَّب على load_iris.

٣. استخدم ColumnTransformer لدمج معالجة عمودين عدديين وعمود فئوي واحد في Pipeline واحد.

حلول مقترحة (تلميحات)

١. Pipeline([('imputer', SimpleImputer()), ('scaler', StandardScaler()), ('model', LogisticRegression())])

٢. from sklearn.metrics import classification_report, confusion_matrix; استدعهما بعد .predict

٣. ColumnTransformer([('num', StandardScaler(), [...]), ('cat', OneHotEncoder(), [...])])

مراجع وروابط رسمية

- التوثيق الرسمي: https://scikit-learn.org/stable/
- دليل المستخدم: https://scikit-learn.org/stable/user_guide.html
- الكود المصدري: https://github.com/scikit-learn/scikit-learn

خاتمة الجزء الأول من Scikit-learn

غطّينا فلسفة التصميم القائمة على الاتساق وEstimator API الموحّد، ومفاهيم fit/predict/transform وPipeline، وأهم أدوات المعالجة المسبقة والتقييم. الجزء الثاني سيتناول أمثلة عملية متقدمة (خطوط أنابيب كاملة من البداية للنهاية)، الأداء، التصحيح، أفضل الممارسات، ومشاريع مصغّرة — كل ذلك مع الحفاظ على التركيز على الواجهة البرمجية لا الخوارزميات.